



IT1244 – Artificial Intelligence: Technology & Impact

AI that Learns – Neural Network

Overview

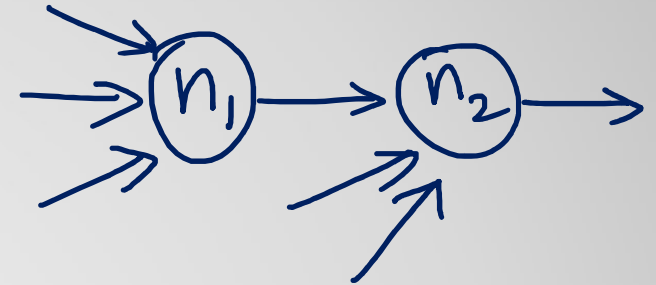
Biological motivation

- What is Neural Network?
- What is a neuron in Neural Network?
- Activation functions used in Neuron
- Single-layer Neural Network
- Multi-layer Neural Network
- Implementing Neural Network in Python (Keras library)

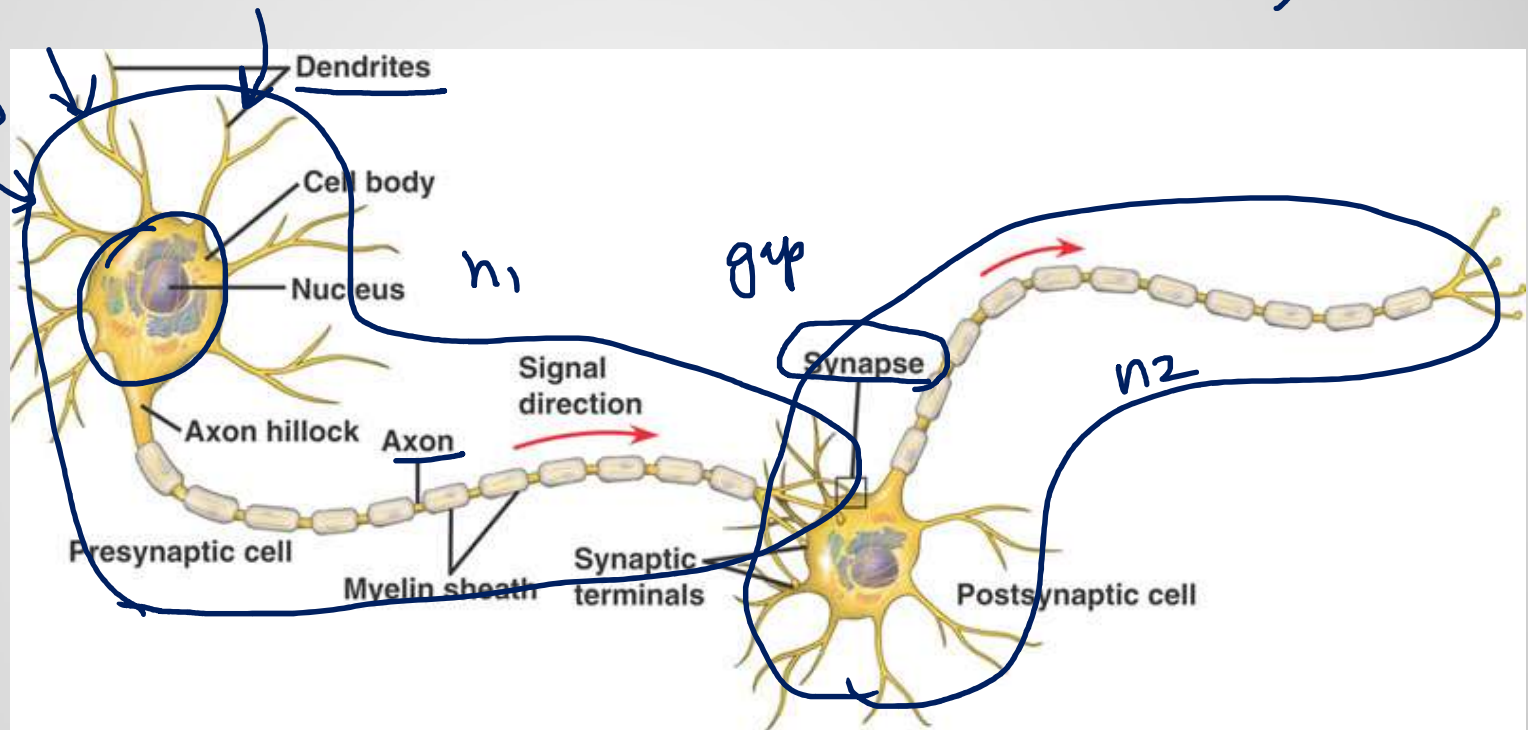
↓
NN + DL

What is Neural Network?

Biological Neural Network



Neurons



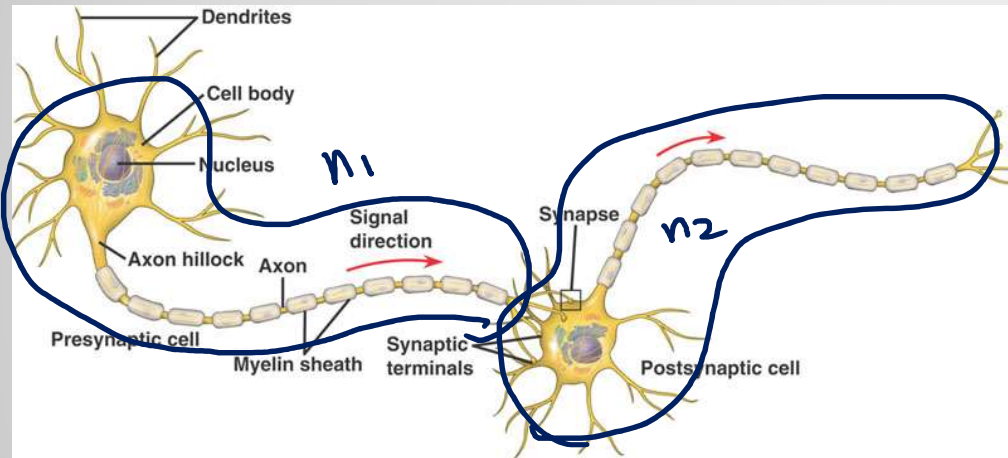
Biological Motivation

- Study of artificial neural network (ANN) has been inspired by the biological learning system: inter-connection of neurons in brain.
- Human brain contains billion of neurons which are connected to many other neurons to form a network
- The brain of an adult human contains around 10^{11} neurons. Each neuron is connected to thousands of other neurons.
- Artificial Neural Networks are built on densely interconnected set of simple units which takes number of real-valued inputs and produce a single real-valued output.

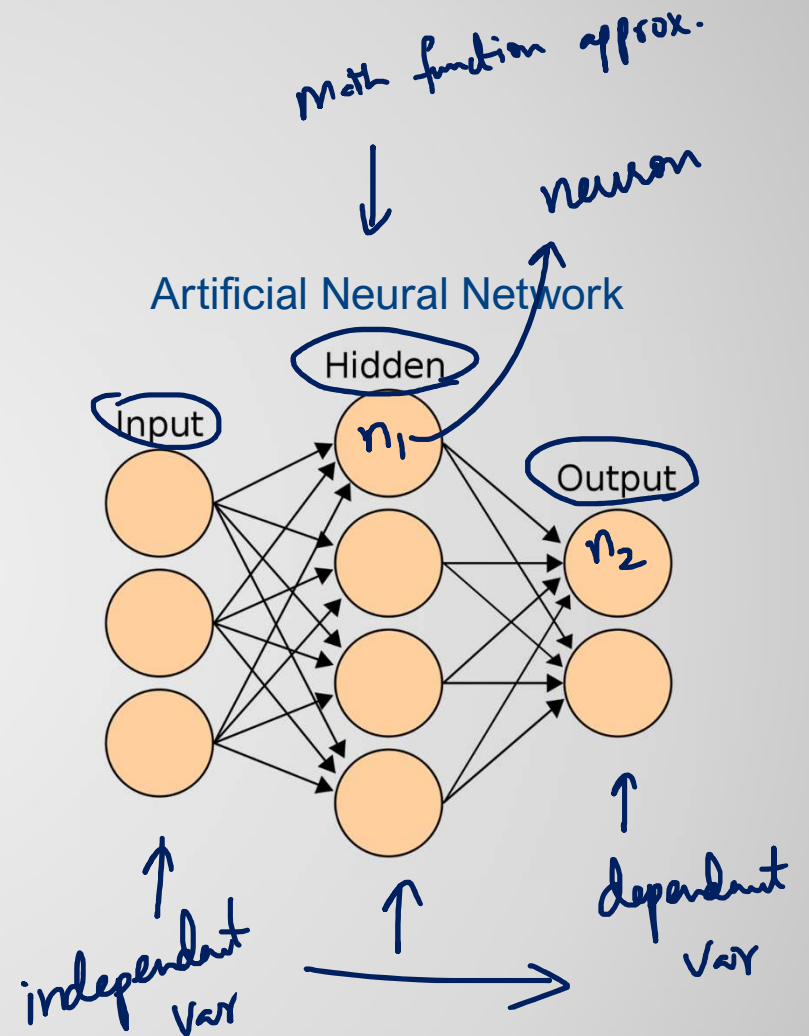


Artificial Neural Network

Biological Neural Network

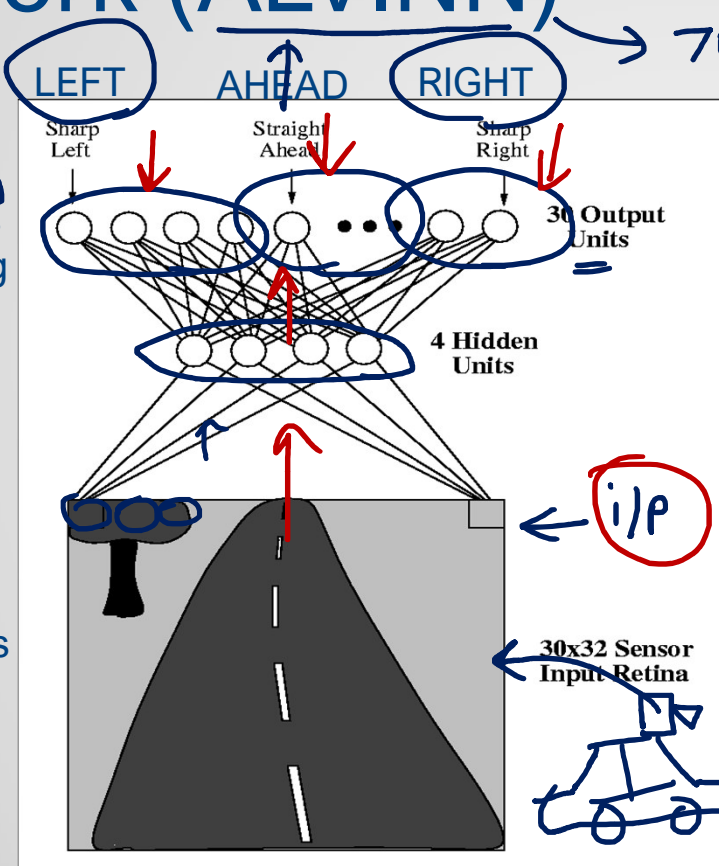


Artificial Neural Network



Autonomous Land Vehicle In a Neural Network (ALVINN) 1989

system setup



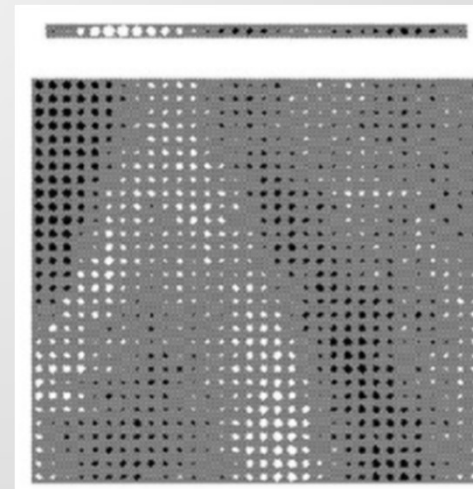
30 outputs for steering

4 hidden units

30 x 32 pixels as inputs

i/p

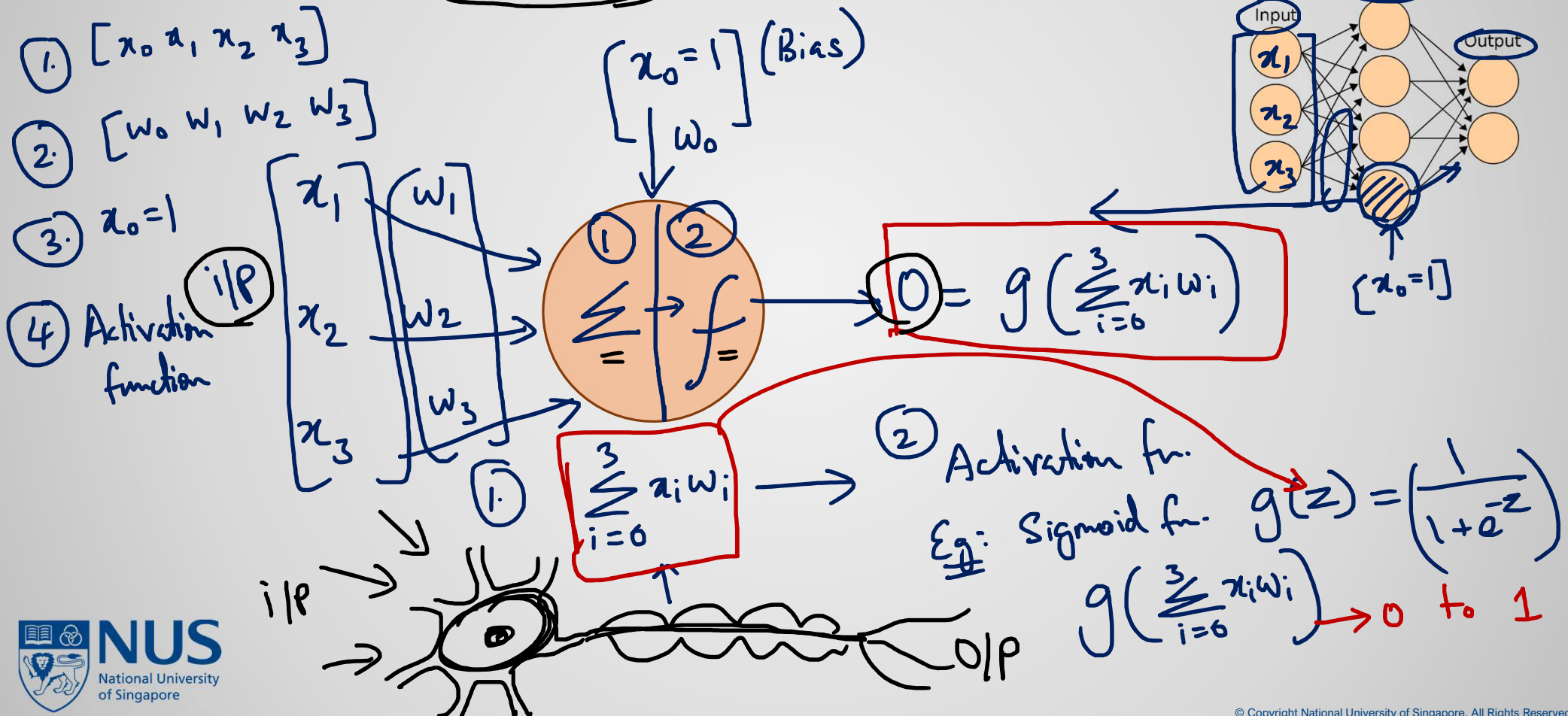
30x32 Sensor Input Retina



30 x 32 weights into one of four hidden unit

What is a **neuron** in Neural Network?

What is a neuron in Neural Network?

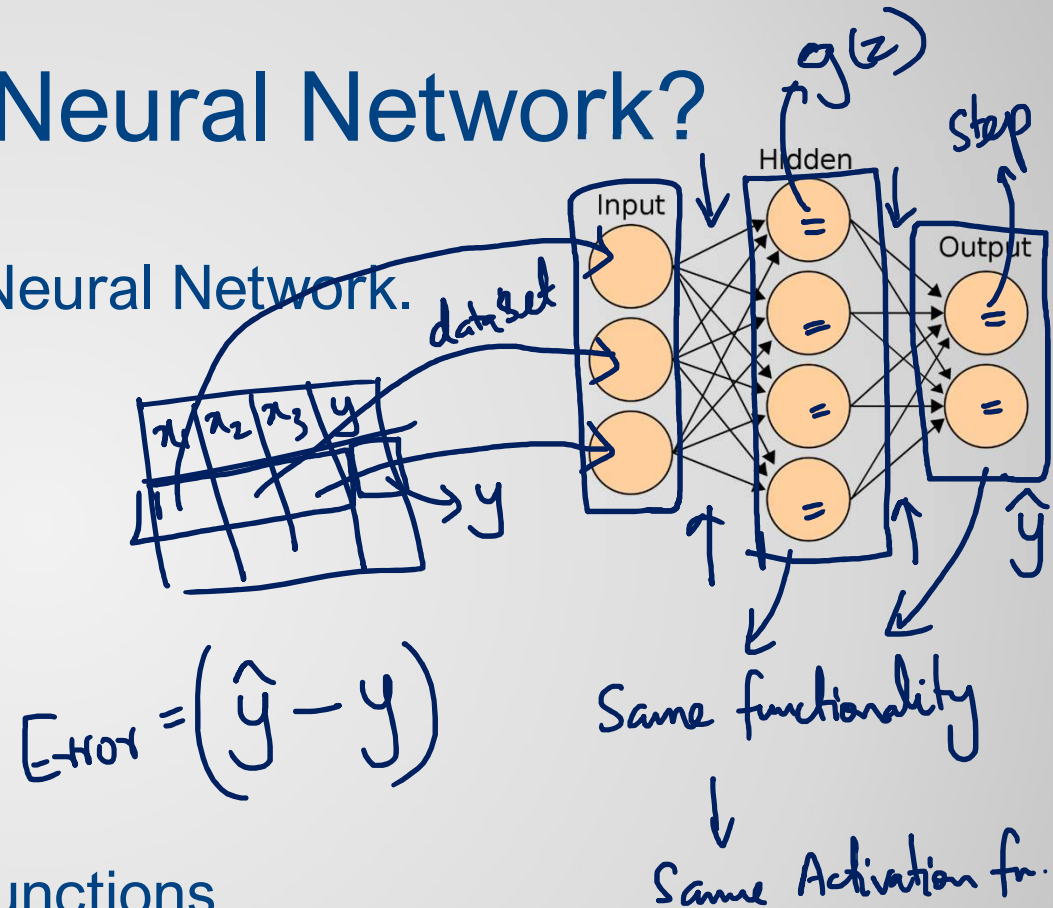


What is a neuron in Neural Network?

- A neuron is building block of Neural Network.

- Each neuron has:

- Set of Inputs
 - One output
 - Weights
 - Activation function
 - Bias $x_0 = 1$
- Red-value*

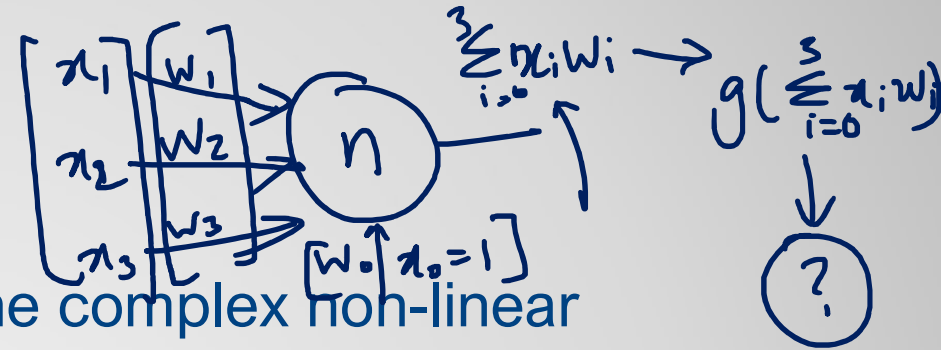


- Each layer has same neuron functions



Activation functions in Neural Network

Activation functions

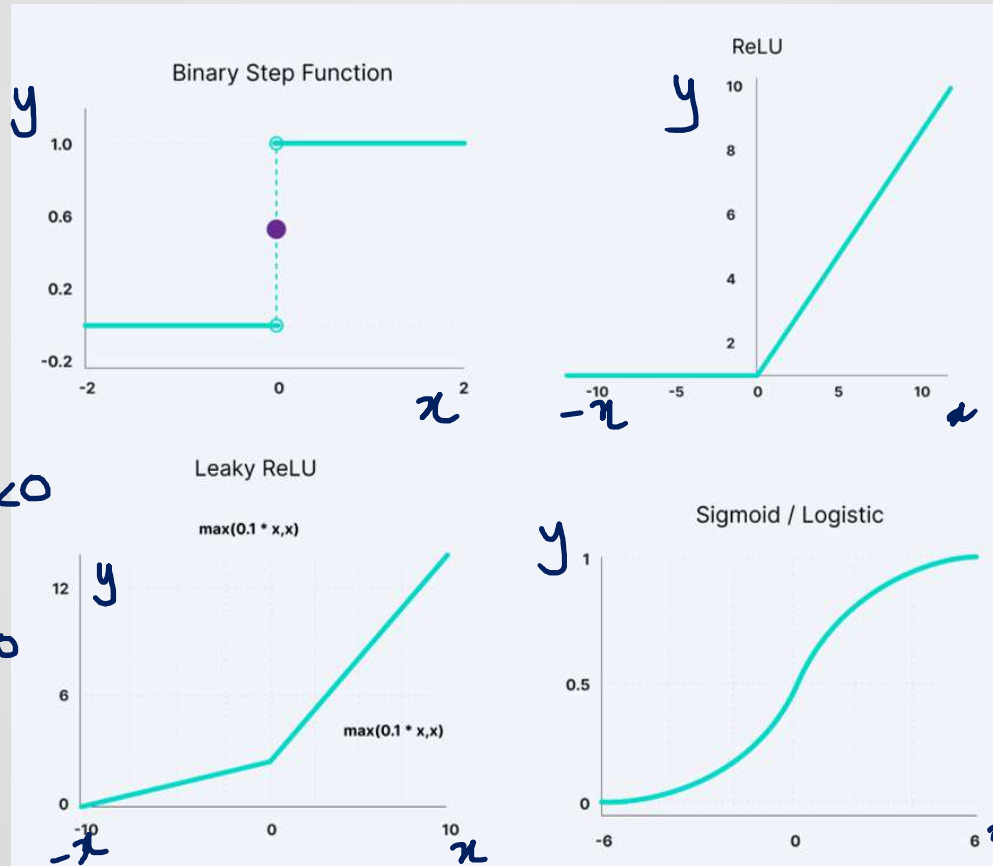


- Activation Function helps to solve the complex non-linear model.
- If only linear combination of x_i and w_i are used, then the neural network will not be able to learn complex data from audio, images, speech, etc.
- Commonly used activation functions are:
 - 1. Step function
 - 2. Sigmoid function
 - 3. ReLu – Rectified Linear Units
 - 4. Leaky ReLu – Leaky Rectified Linear Units

Keys

Activation functions

$$y = \begin{cases} 0 & x < 0 \\ 1 & x \geq 0 \end{cases}$$



$$y = \begin{cases} 0 & x < 0, \\ x & x \geq 0. \end{cases}$$

$$y = \begin{cases} 0.01 * x & x < 0 \\ x & x \geq 0 \end{cases}$$

$$y = g(x) = \frac{1}{1 + e^{-x}}$$

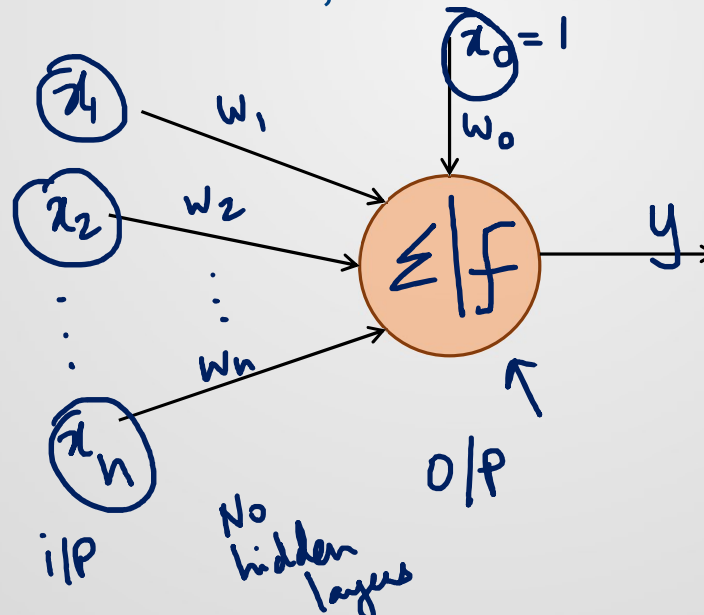
→ Source: <https://www.v7labs.com/blog/neural-networks-activation-functions>

Single-layer Neural Network (Perceptron)



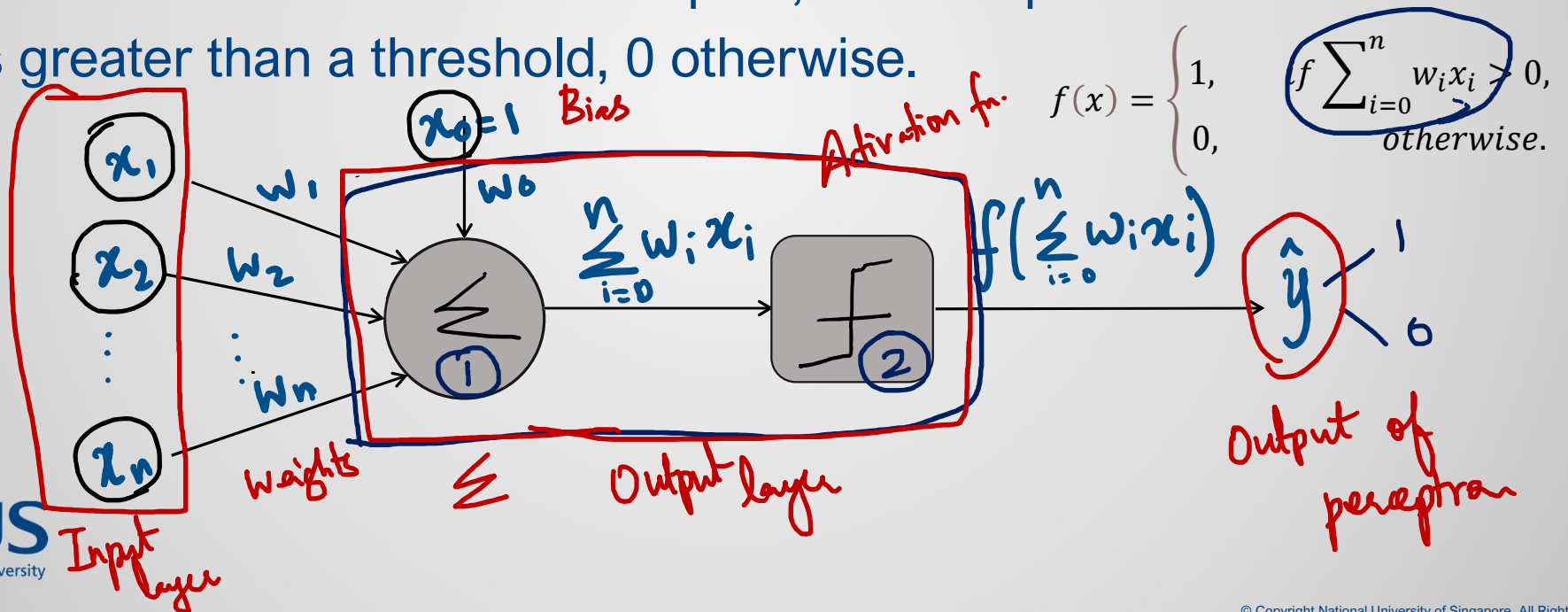
What is Perceptron?

- A perceptron takes a vector of real-valued inputs, calculates a linear combination of these inputs, then outputs a 1 if the result is greater than a threshold, 0 otherwise.



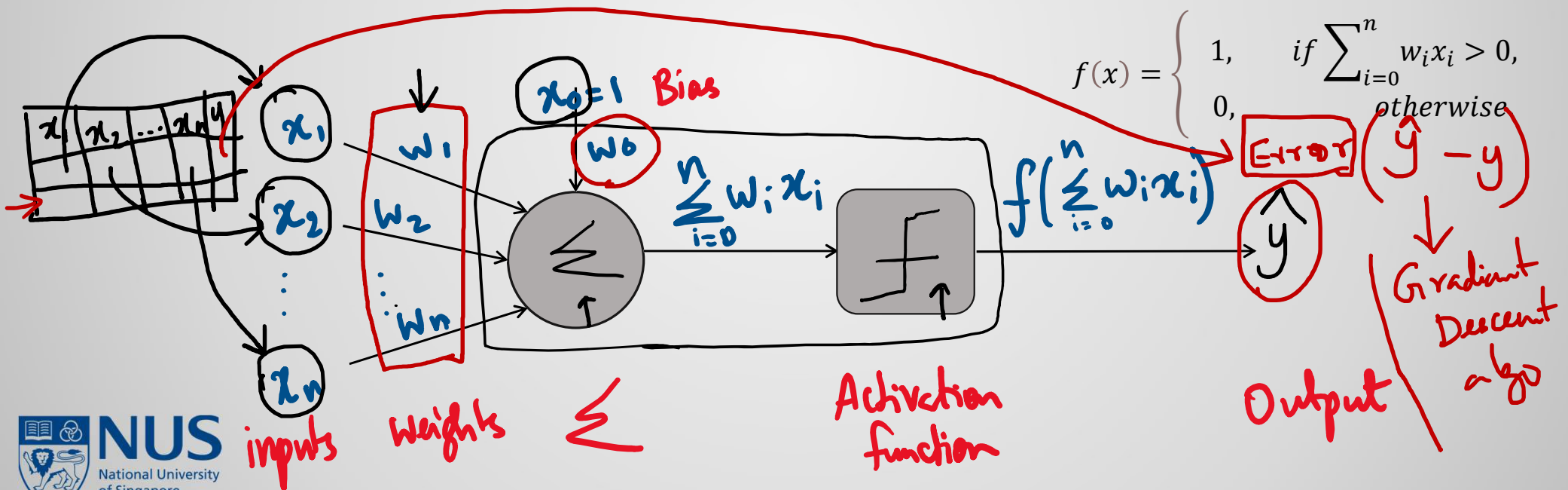
What is Perceptron?

- A perceptron takes a vector of real-valued inputs, calculates a linear combination of these inputs, then outputs a 1 if the result is greater than a threshold, 0 otherwise.



Training a Perceptron

- The weights are adjusted based on the expected output $\hat{y}$ and the actual output y .

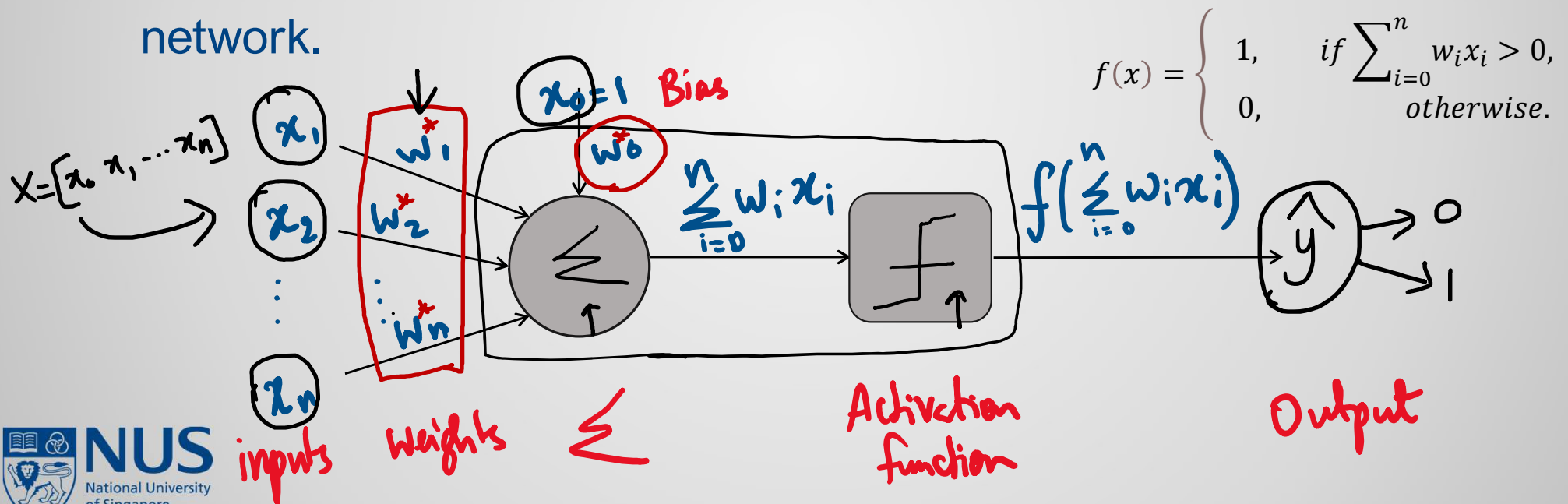


Class prediction in Perceptron

$$\hat{y} = \underset{\uparrow}{w_0} + \underset{\uparrow}{w_1} x$$

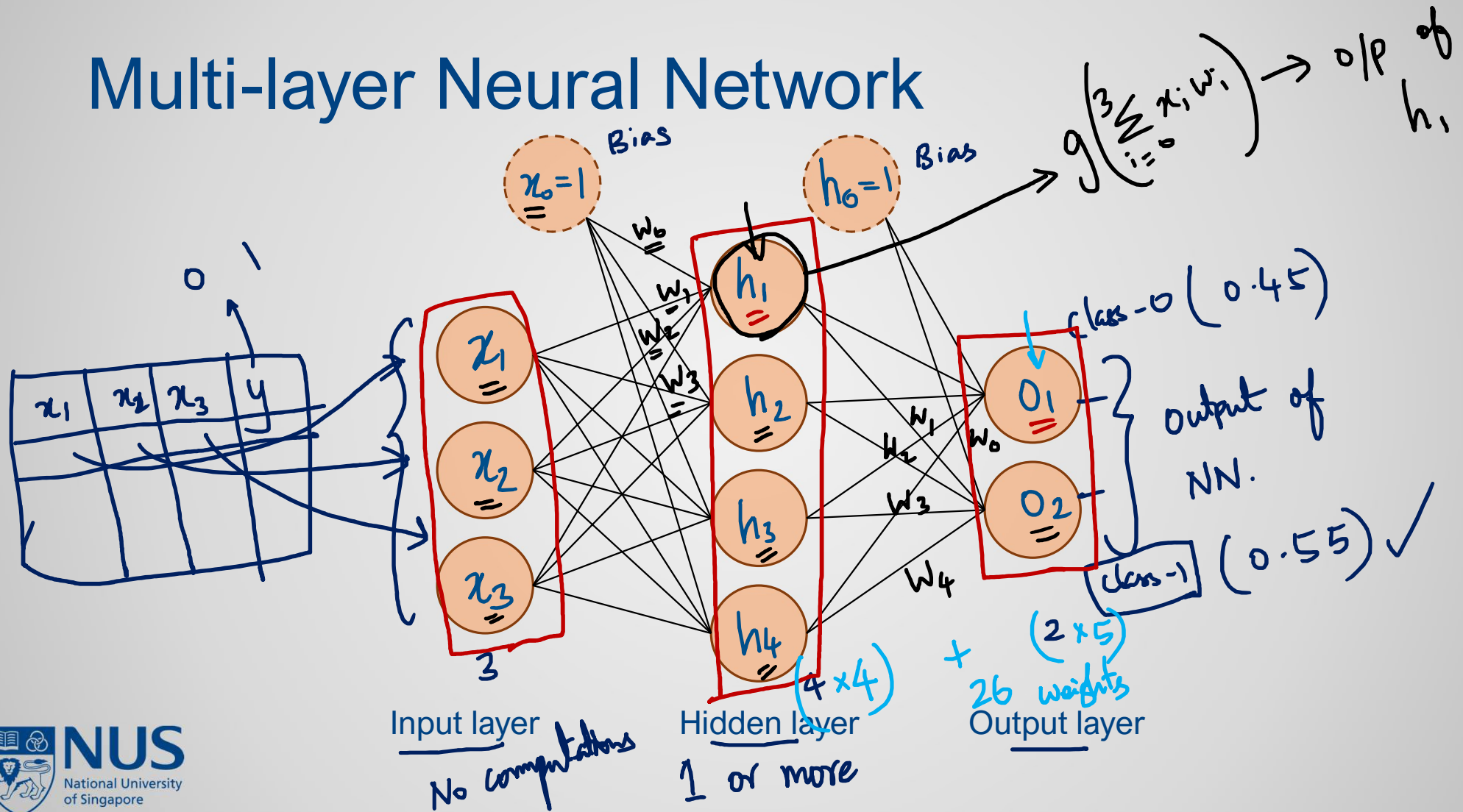
$$W^* = [w_0^*, w_1^*, \dots, w_n^*] \leftarrow$$

- After finding the optimal parameters, the output class is predicted by substituting the learned weights W^* in the neural network.



Multi-layer Neural Network

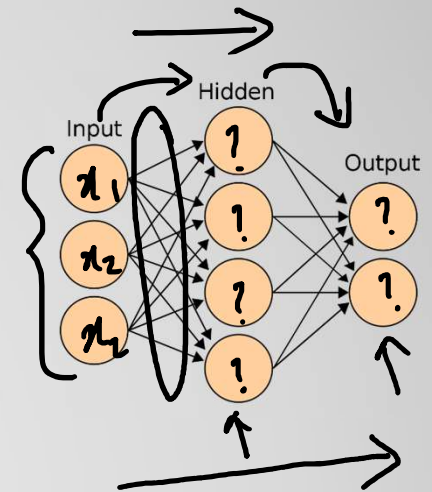
Multi-layer Neural Network



Feedforward in Neural Network

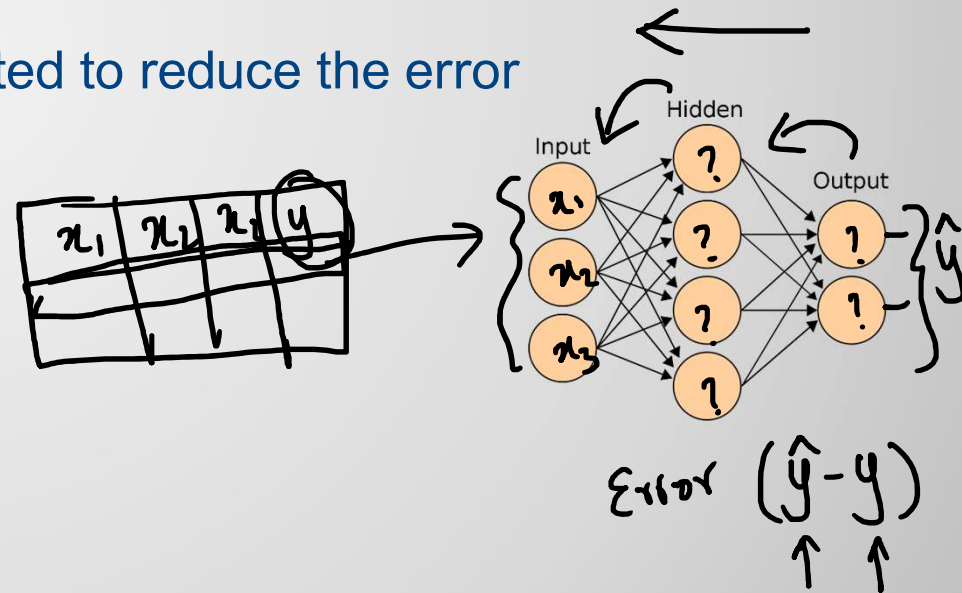
Working forward through the network:

- Get one labeled training data
- Plug the data into the input layer
- Compute the values for the hidden layer using the input layer and the weights
- Compute the values for the output layer using the hidden layer as input and its weights
- The output layer is the output of the neural network



Backpropagation in Neural Network

- Learning Algorithm:
 - we cycle through our training samples
 - If the output of the network is correct, no changes are made to the weights
 - If there is an error, weights are adjusted to reduce the error



Backpropagation in Neural Network

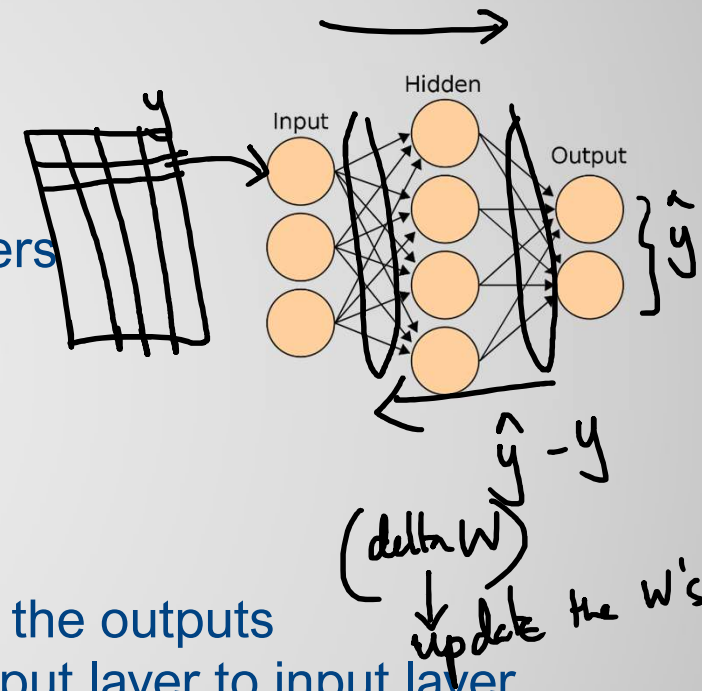
Backpropagation Learning Algorithm:

- Initialize random values for the weights in all layers

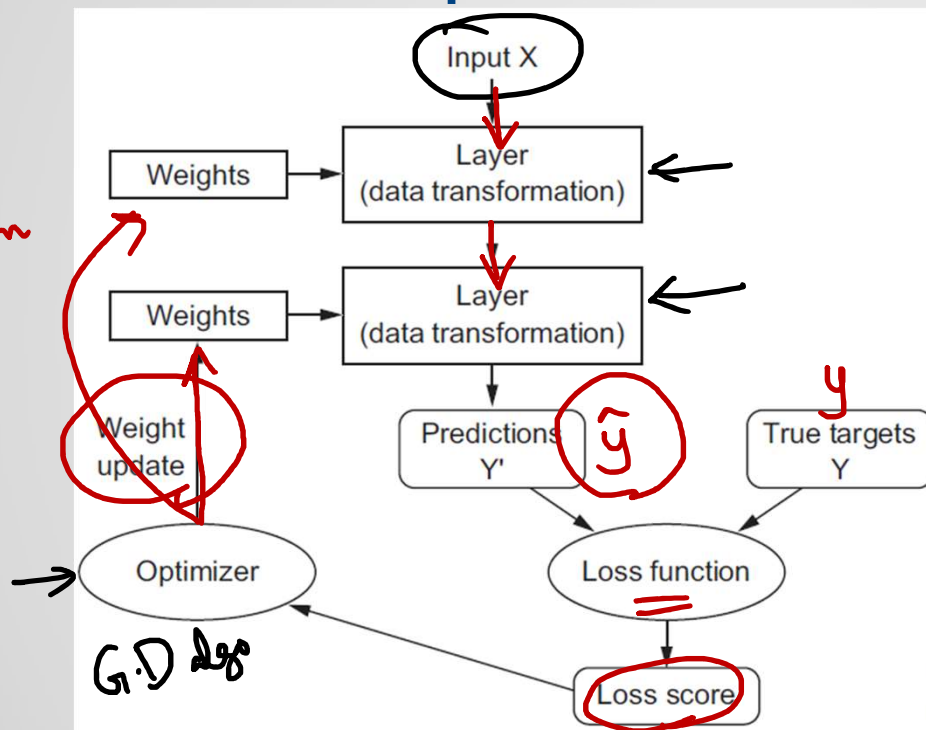
- Repeat until convergence:

→ for each training datapoint ($\mathbf{x}$, y)

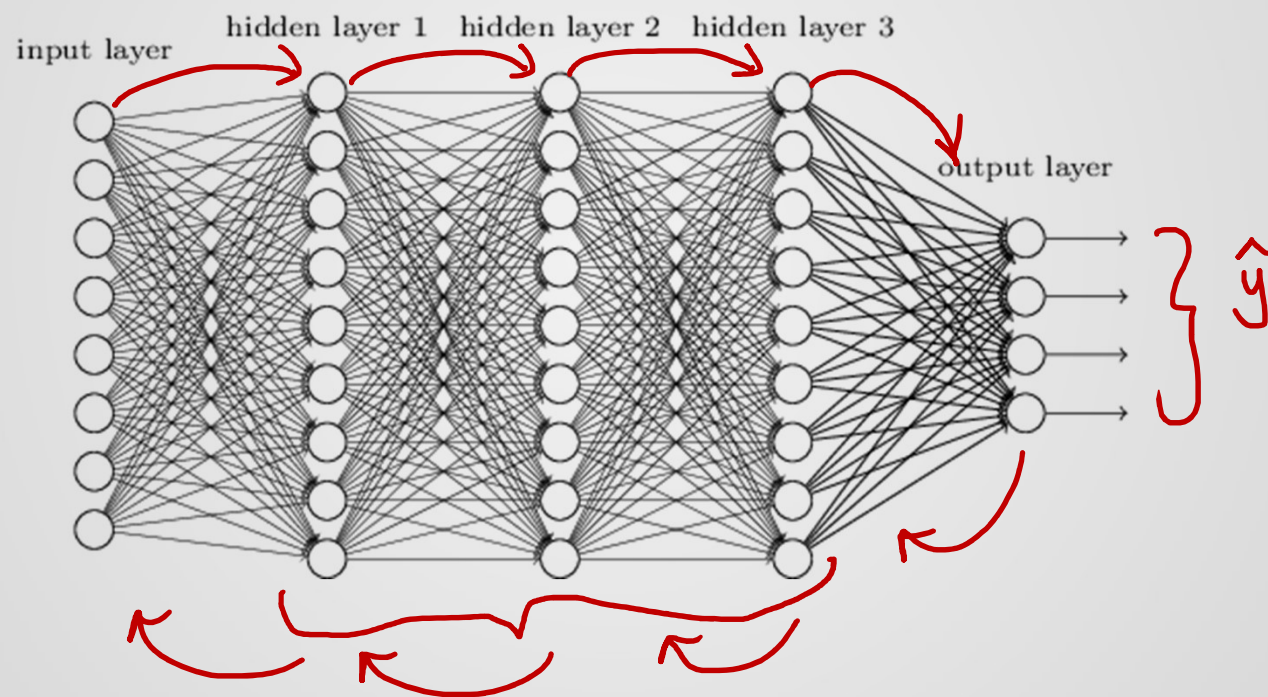
- Propagate the inputs forward to compute the outputs
- Propagate the deltas backwards from output layer to input layer
- Update every weight in network using deltas



Relationship between input, network layers, loss function, optimizers and output



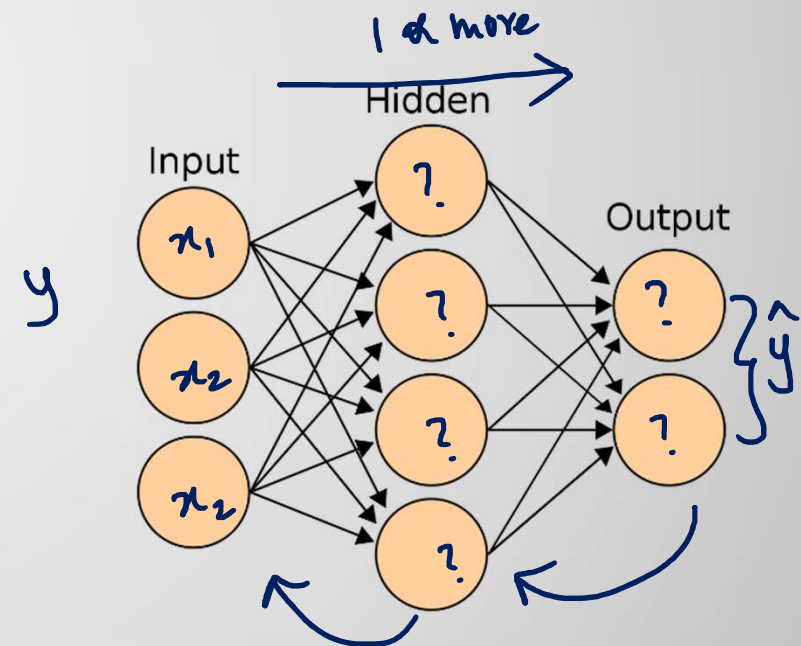
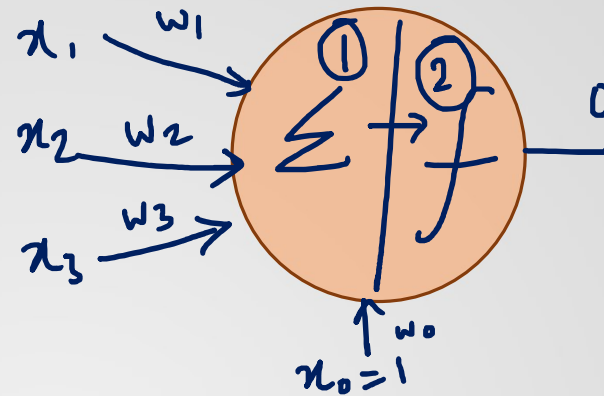
More hidden layers



Summary

Summary

- ① Human brain – neurons
Biological NN vs Artificial NN
- ② Structure of node in NN
- ③ Activation functions → *model non-linear data*
- ④ Single-layer Neural Network → *Perceptron*
- ⑤ Multi-layer Neural Network
 - Feedforward in NN
 - Backpropagation in NN



Neural Network – Python (Keras library)

NN and DL

TensorFlow

Neural Network Layers

```

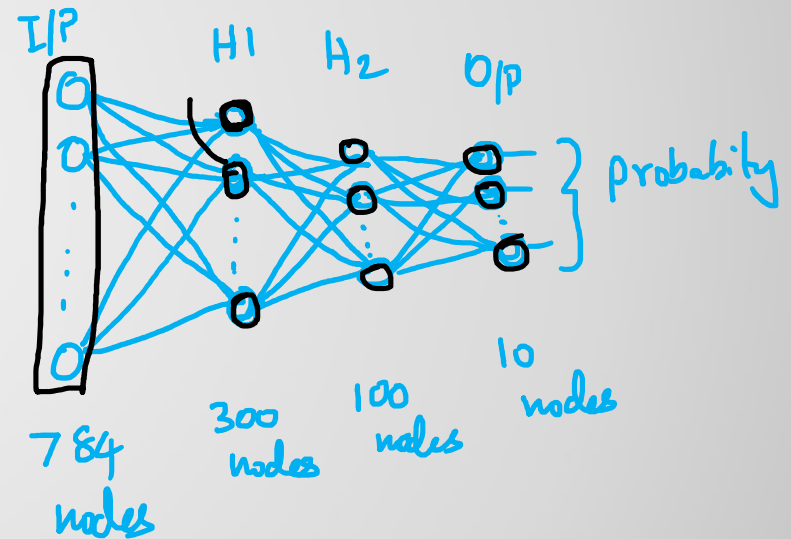
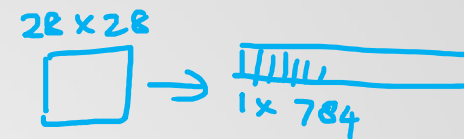
12 model = keras.models.Sequential()
13 model.add(keras.layers.Flatten(input_shape=[28, 28]))
14 model.add(keras.layers.Dense(300, activation="relu"))
15 model.add(keras.layers.Dense(100, activation="relu"))
16 model.add(keras.layers.Dense(10, activation="softmax"))
17 model.summary()
    
```

Handwritten annotations:
 - "input" next to `input_shape`
 - "Hidden" next to the first `Dense` layer
 - "Hidden" next to the second `Dense` layer
 - "output" next to the `softmax` layer
 - "multi class → prob." next to the `softmax` layer

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 300)	→ 235500
dense_1 (Dense)	(None, 100)	→ 30100
dense_2 (Dense)	(None, 10)	→ 1010
Total params: 266,610		
Trainable params: 266,610		
Non-trainable params: 0		

Handwritten annotations:
 - "i/p" with an arrow pointing to the `flatten` layer
 - "H₁" next to the first `dense` layer
 - "H₂" next to the second `dense` layer
 - "O/P" next to the third `dense` layer
 - "multi class → prob." next to the `softmax` layer



$$[784 \times 300] + 300 = 235500$$

